

tf.compat.v1.case Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/case

Create a case operation.

Function Signatures

```
tf.compat.v1.case( pred_fn_pairs, default=None,
exclusive=False, strict=False, name='case' )
```

Code Examples

```
tf.compat.v1.case(
pred_fn_pairs,
default=None,
exclusive=False,
strict=False,
name='case'
)
```

```
tf.compat.v1.case(  
    pred_fn_pairs,  
    default=None,  
    exclusive=False,  
    strict=False,  
    name='case'  
)
```

```
if (x < y) return 17;  
else return 23;
```

```
if (x < y) return 17;  
else return 23;
```

```
f1 = lambda: tf.constant(17)  
f2 = lambda: tf.constant(23)  
r = tf.case([(tf.less(x, y), f1)], default=f2)
```

```
f1 = lambda: tf.constant(17)  
f2 = lambda: tf.constant(23)  
r = tf.case([(tf.less(x, y), f1)], default=f2)
```

```
if (x < y && x > z) raise ValueError("Only one predicate may evaluate  
to True");  
if (x < y) return 17;  
else if (x > z) return 23;  
else return -1;
```

```
if (x < y && x > z) raise ValueError("Only one predicate may evaluate  
to True");  
if (x < y) return 17;  
else if (x > z) return 23;  
else return -1;
```

Documentation

Create a case operation.

See also `tf.switch_case`.

The `pred_fn_pairs` parameter is a dict or list of pairs of size `N`.

Each pair contains a boolean scalar tensor and a python callable that creates the tensors to be returned if the boolean evaluates to `True`.

`default` is a callable generating a list of tensors. All the callables in `pred_fn_pairs` as well as `default` (if provided) should return the same number and types of tensors.

If `exclusive==True`, all predicates are evaluated, and an exception is thrown if more than one of the predicates evaluates to `True`.

If `exclusive==False`, execution stops at the first predicate which evaluates to `True`, and the tensors generated by the corresponding function are returned immediately. If none of the predicates evaluate to `True`, this operation returns the tensors generated by default.

`tf.case` supports nested structures as implemented in

`tf.nest`. All of the callables must return the same (possibly nested) value structure of lists, tuples, and/or named tuples. Singleton lists and tuples form the only exceptions to this: when returned by a callable, they are implicitly unpacked to single values. This behavior is disabled by passing `strict=True`.

If an unordered dictionary is used for `pred_fn_pairs`, the order of the conditional tests is not guaranteed. However, the order is guaranteed to be deterministic, so that variables created in conditional branches are created

in fixed order across runs.

Example 1:

Pseudocode:

`## Expressions:`

Example 2:

Pseudocode:

`## Expressions:`

Returns

`## Raises`

eager compatibility

Unordered dictionaries are not supported in eager mode when `exclusive=False`.
Use a list of tuples instead.